

Project AIS – AI-Based Search for Vendor Transaction Data

1. Problem Statement

Project AIS aims to solve the challenge of efficiently retrieving and analyzing contract-related information from a database. Organizations often struggle with managing vendor contracts, invoices, purchase orders, and financial records, leading to inefficiencies in contract management and financial tracking. The project provides an intelligent system to query and retrieve contract details using natural language processing.

As a Vendor Manager or Business Owner, connecting to a database, querying transaction data is tedious, or opening a SaaS-based application and applying multiple filters to extract simple information is time-consuming.

2. Objective

The primary objective of Project AIS is to develop an AI-driven contract intelligence system that allows users to query contract-related data using natural language. The system should:

- Enable efficient contract and invoice retrieval.
- Provide accurate financial insights.
- Reduce manual effort in contract analysis.
- Enhance decision-making with structured data outputs.

3. Approach

The approach taken for Project AIS involves:

1. **Database Structuring** – Organizing contract data into an SQLite database.
2. **Natural Language Processing** – Using LlamaIndex for querying structured data.
3. **AI-Powered Query Handling** – Leveraging OpenAI's GPT models to process user queries.
4. **Data Indexing and Retrieval** – Structuring contract information for faster access.
5. **Response Synthesis** – Generating meaningful, structured responses from raw database results.

4. Technical Overview

Technology Stack:

- **Programming Language:** Python
- **Database:** SQLite
- **AI Model:** LlamaIndex, OpenAI GPT-3.5 Turbo

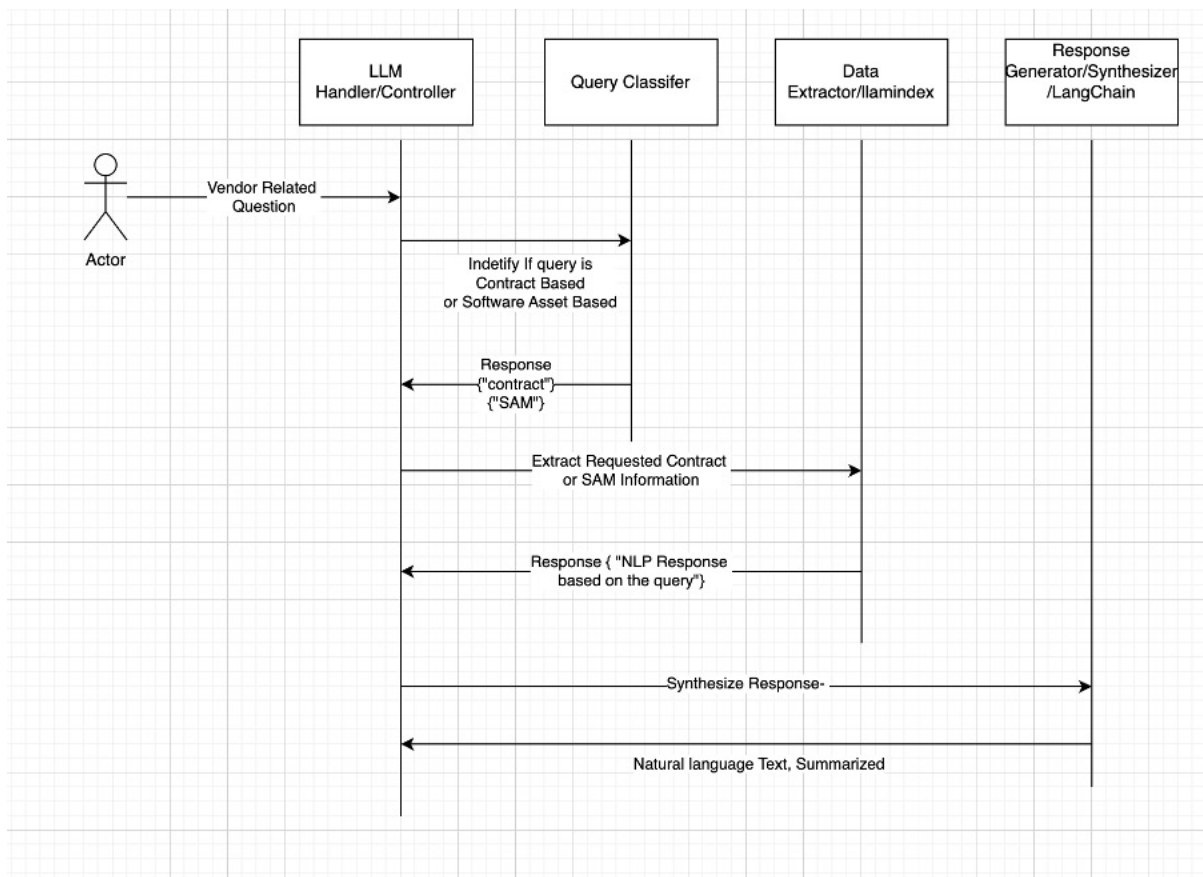
- **Libraries & Frameworks:**
 - LangChain – For managing LLM-based interactions
 - SQLAlchemy (Database ORM)
 - LlamaIndex (NLP Query Processing)
 - Pandas (Data Handling)

5. System Design

Modular Design:

- **Database Layer:** Stores contract, invoice, and vendor information.
- **Query Processing Layer:** Handles user queries and translates them into structured SQL queries.
- **AI Model Layer:** Uses LlamaIndex and OpenAI GPT for interpretation.
- **Response Handler:** Formats and synthesizes the response for the end user.

Data Flow:



1. User enters a query in natural language.
2. Query is classified into different types (e.g., total contract value, invoice details, active contracts, etc.).
3. The relevant SQL query is generated and executed on the SQLite database.
4. Results are fetched and structured into an AI-generated response.

5. The response is displayed to the user.

6. Core Components

1. LLM Handler

Detailed Summary

The LLM Handler module acts as the central interface for handling all Large Language Model (LLM) interactions within the AIS system. It is responsible for processing user queries, invoking the appropriate LLM models, and formatting responses in a structured manner. This module ensures seamless communication between the query classification, data extraction, and response synthesizer components by coordinating the flow of data.

Key Responsibilities:

1. **Handling User Queries** – Receives user input and determines whether it requires contract-related, software-related, or general AI-based processing.
2. **Invoking LLM (GPT-4 via LangChain)** – Uses LangChain's ChatOpenAI to query OpenAI's GPT-4 and generate contextually accurate responses.
3. **Processing and Formatting Data** – Ensures that the retrieved contract/software information is structured before being passed to the Response Synthesizer.
4. **Integrating with Other Modules** – Works alongside Query Classifier (to categorize queries) and LlamaIndex-based Data Extractor (to fetch contract/software details).

2. Query Classifier

The Query Classifier determines the category of a given user query by leveraging sentence embeddings and similarity matching. It pre-stores embeddings of predefined query categories (e.g., "contract," "software," and "other") in Redis for quick retrieval. When a new query arrives, the classifier computes its embedding, compares it with stored embeddings, and assigns the best-matching category. This improves query routing efficiency and reduces unnecessary calls to downstream systems.

3. Data Extractor (Llama Index)

The Data Extractor uses Llama Index to fetch structured data from an SQL database based on natural language queries. It connects to the SQLite database, ensures the required tables (like contracts) exist, and creates an index for efficient querying. Unlike traditional SQL queries, Llama Index allows users to retrieve relevant information using conversational search without manually writing SQL. To improve efficiency, Llama Index should ideally be preloaded rather than being initialized for every query.

4. Response Synthesizer

The Response Synthesizer module generates concise and relevant responses to user queries by summarizing contract details retrieved from a database or a model. It takes structured data or model-generated responses and formats them into human-readable text using an LLM-powered summarization prompt. If contract details are missing, it returns an appropriate fallback message. The module ensures that responses are clear, non-repetitive, and directly answer the user's question.

7. Implementation Details

Project Structure:

1. Database Module (db/):

- `ais.db` – SQLite database containing contract data.
- `db_conn.py` – Handles database connection.
- `db_create_contract_data.py` – Creates tables and schema.
- `fetch_contract_details.py` – Fetches relevant contract data.
- `insert_sample_data.py` – Seeds the database with sample contract records.

2. LLM Processing Module (llm/):

- `llama_contract_index.py` – Creates and maintains the LlamaIndex structure.
- `llm_handler.py` – Handles interaction between queries and the AI model.
- `query_classifier.py` & `query_classifier2.py` – Classifies user queries into predefined categories.
- `response_synthesizer.py` – Converts raw query results into meaningful responses.
- `vendor_name_extractor.py` – Identifies vendor names from user queries.

3. Utility Module (utils/):

- `config.py` – Stores configuration settings.

4. Main Execution File:

- `llm_handler.py` – Entry point to run the project and interact with the system.

8. Test Results

The system has been tested for different contract-related queries, including:

1. General Vendor Information: Successfully retrieves vendor details.
2. Active Contracts: Filters and fetches ongoing contracts.
3. Invoices Paid Till Now: Returns total invoice payments made.

4. Number of Purchase Orders: Counts PO records associated with a vendor.
5. Total Contract Value: Aggregates the total contract amount.
6. Active Contract Value: Filters and calculates the total active contract value.

Test cases verified that the system provides accurate and timely responses.

9. GitHub Repository

[GitHub Repo](#)

10. Next Steps

Future improvements for Project AIS include:

- Enhancing NLP Accuracy: Fine-tuning AI models for better query understanding.
- Expanding Database Coverage: Adding more contract metadata.
- Integrating with External APIs: Connecting with ERP and finance tools.
- Implementing a Web UI: Providing a frontend for user interaction.
- Optimizing Query Performance: Using caching and indexing techniques.